# Database and Application Security

# Database Security

- **Database Security** - protection from malicious attempts to steal (view) or modify data.

# Importance of Data

- Bank/Demat accounts
- Credit card, Salary, Income tax data
- University admissions, marks/grades
- Land records, licenses
- Data = crown jewels for organizations
- Recent headlines:
  - Personal information of millions of credit card users stolen
    - Laws on privacy in the US
    - Theft of US data in India
  - Criminal gangs get into identity theft

# Identity Theft

- Pretend to be someone else and get credit cards/loans in their name
  - Identification based on “private” information that is not hard to obtain online
- Harder to catch criminals
- Hurts victims even more than regular theft
  - Blame goes on innocent people to prove they didn’t get loans or make credit card payment
  - Credit history gets spoilt, making it harder to get future loans
  - And you may have been robbed without ever knowing about it.

# What to worry?

- Most Web applications vulnerable to SQL injection attacks

# Levels of Data Security

- Human level: Corrupt/careless User
- Network/User Interface
- Database application program
- Database system
- Operating System
- Physical level

# Physical/OS Security

- Physical level
  - Traditional lock-and-key security
  - Protection from floods, fire, etc.
    - E.g. WTC (9/11), fires in IITM, WWW conf website, etc.
  - Protection from administrator error
    - E.g. delete critical files
  - Solution
    - Remote backup for disaster recovery
    - Plus archival backup (e.g. DVDs/tapes)
- Operating system level
  - Protection from virus/worm attacks critical

# Database Encryption

- E.g. What if a laptop/disk/USB key with critical data is lost?
- Partial solution: encrypt the database at storage level, transparent to application
  - Whole database/file/relation
    - Unit of encryption: page
  - Column encryption
- Main issue: key management
  - E.g. user provides decryption key (password) when database is started up
- Supported by many database systems
  - Standard practice now to encrypt credit card information, and other sensitive information

# Security (Cont.)

- Network level: must use encryption to prevent
  - Eavesdropping: unauthorized reading of messages
  - Masquerading:
    - pretending to be an authorized user or legitimate site, or
    - sending messages supposedly from authorized users

# Network Security

- All information must be encrypted to prevent eavesdropping
  - Public/private key encryption widely used
  - Handled by secure http - https://
- Must prevent person-in-the-middle attacks
  - E.g. someone impersonates seller or bank/credit card company and fools buyer into revealing information
    - Encrypting messages alone doesn't solve this problem
    - More on this in next slide

# Site Authentication

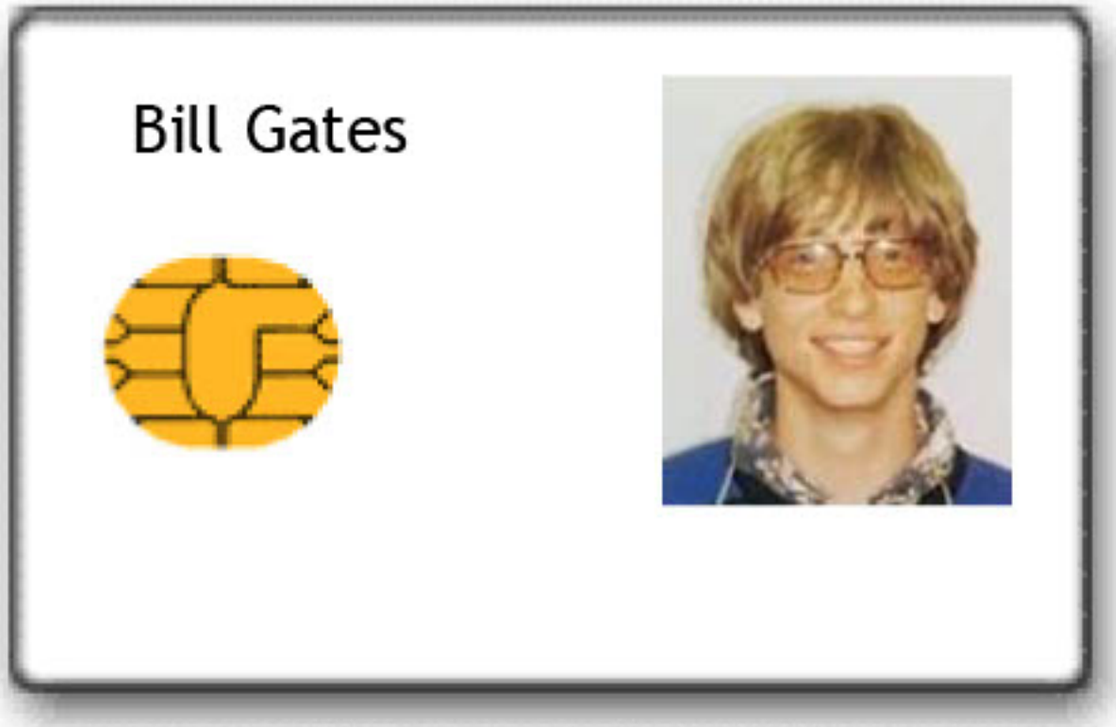
- Digital certificates are used in https to prevent impersonation/man-in-the middle attack
  - Certification agency creates digital certificate by encrypting, e.g., site's public key using its own private key
    - Verifies site identity by external means first!
  - Site sends certificate to buyer
  - Customer uses public key of certification agency to decrypt certificate and find sites public key
    - Man-in-the-middle cannot send fake public key
  - Sites public key used for setting up secure communication

# Security at the Database/Application Program

- Authentication and authorization mechanisms to allow specific users access only to required data
- Authentication:  who are you? Prove it!
- Authorization:  what you are allowed to do

# User Authentication

- Password
  - Most users abuse passwords. For e.g.
    - Easy to guess password
    - Share passwords with others
- Smartcards
  - Need smartcard
  - + a PIN or password

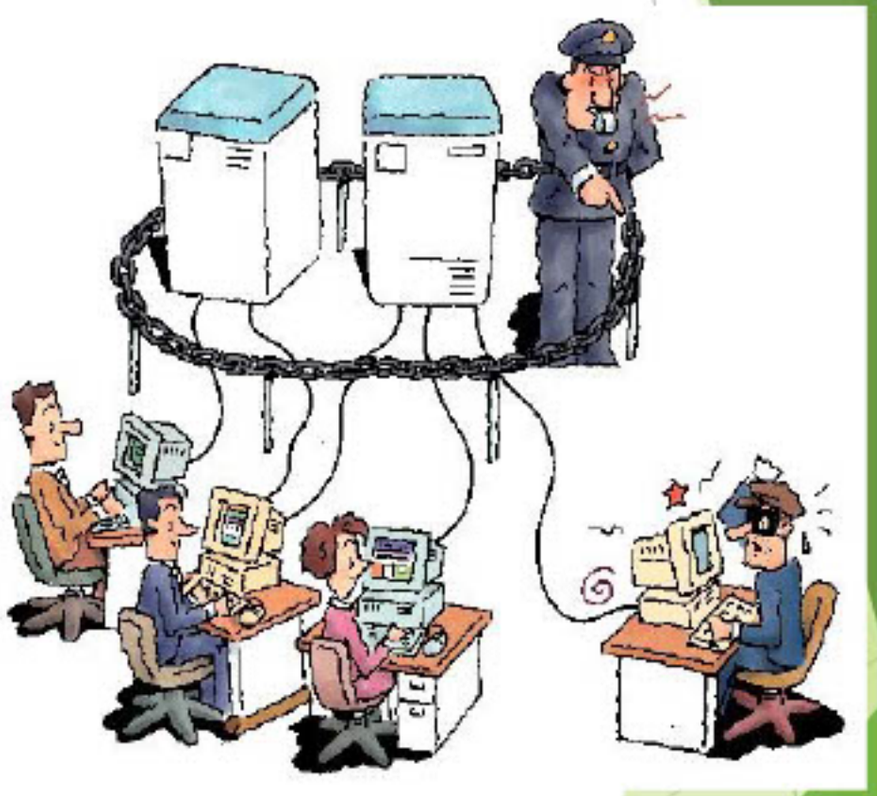


# User Authentication

- Central authentication systems allow users to be authenticated centrally
  - LDAP or MS Active Directory often used for central authentication and user management in organizations
- Single sign-on: authenticate once, and access multiple applications without fresh authentication
  - Microsoft passport, PubCookie etc
  - Avoids plethora of passwords
  - Password only given to central site, not to applications

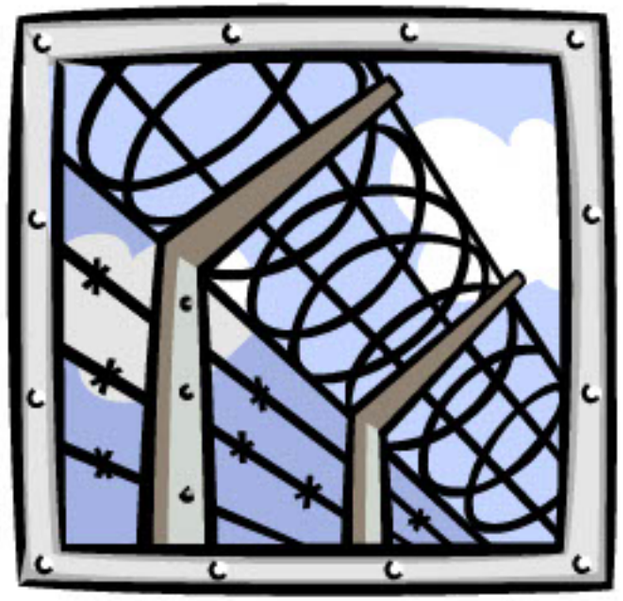
# Authorization

- Different authorizations for different users
  - Accounts clerk vs.
  - Accounts manager vs.
  - End users



# Database/Application Security

- Ensure that only authenticated users can access the system
- And can access (read/update) only data/interfaces that they are authorized to access



# Limitations of SQL Authorization

- SQL does not support authorization at a tuple level
  - E.g. we cannot restrict students to see only (the tuples storing) their own grades
- Web applications are dominant users of databases
  - Application end users don't have database user ids, they are all mapped to the same database user id
  - Database access control provides only a very coarse application-level access control

# Access Control in Application Layer

- Applications authenticate end users and decide what interfaces to give to whom
  - Screen level authorization: which users are allowed to access which screens
  - Parameter checking: users only authorized to execute forms with certain parameter values
    - E.g. CSE faculty can see only CSE grades

# Privacy

- Aggregate information about private information can be very valuable
  - E.g. identification of epidemics, mining for patterns (e.g. disease causes) etc.
- Privacy preserving data release
  - E.g. in US, many organizations released "anonymized" medical data, with names removed, but zipcode (= pincode), sex and date of birth retained
    - Turns out above (zipcode,sex,date of birth) uniquely identify most people!
      - Correlate anonymized data with (say) electoral data with same information
  - Recent problems at America Online
    - Released search history, apparently anonymized, but users could be easily identified in several cases
      - Several top officials were fired
  - Earlier problems revealed medical history of Massachusetts state governer.
- Not yet a criminal issue, but lawsuits have happened
- Conflict with Right To Information Act
  - Many issues still to be resolved

# Application Security

- Applications are often the biggest source of insecurity
  - Poor coding of application may allow unauthorized access
  - Application code may be very big, easy to make mistakes and leave security holes
  - Very large surface area
    - Used in fewer places
      - Some security by obfuscation
      - Lots of holes due to poor/hasty programming

# OWASP Top 10 Web Security Vulnerabilities

1. Unvalidated input
2. Broken access control
3. Broken account/session management
4. Cross-site scripting (XSS) flaws
5. Buffer overflows
6. (SQL) Injection flaws
7. Improper error handling
8. Insecure storage
9. Denial-of-service
10. Insecure configuration management

# SQL Injection (Further Reading)

- E.g. application takes accnt_number as input from user and creates an SQL query as follows:
  - string query = "select balance from account where account_number =‘" + accnt_number +"‘"
  - Suppose instead of a valid account number, user types in
    - ‘; delete from r;

    then (oops!) the query becomes

    select balance from account where account_number =‘ ‘; delete from r;
- Hackers can probe for SQL injection vulnerability by typing, e.g. ‘*** in an input box
  - Tools can probe for vulnerability
  - Error messages can reveal information to hacker

# Preventing SQL Injection (Further Reading)

- To prevent SQL injection attacks use prepared statements (instead of creating query strings from input parameters)
  - PreparedStatement pstmt= conn.prepareStatement(
    "select balance from account where account_number =?“);
    pstmt.setString(1,accnt_number);
    pstmt.execute();
    - (assume that conn is an already open connection to the database)
- Alternatives:
  - use stored procedures
  - use a function that removes special characters (such as quotes) from strings

# Protecting from Users

- Multi-person approval:
  - Standard practice in banks, accounts departments
  - Encoded as part of application workflow
  - External paper trail
- Strong authentication of users
  - Smart cards
- Careful allocation of authorizations on a need to use basis
  - Practical problem: absence of a user should not prevent organization from functioning
  - Many organizations therefore grant overly generous authorizations

# Protecting from Programmers/DBA

- Have password to database, can update anything!
  - Digital signatures by end users can help in some situations
    - E.g. low update rate data such as land records, birth/death data
- Application program has database password
  - Seize control of the application program ➔ can do anything to the database
  - Solution:
    - Don't give database password to development team
    - keep password in a configuration file on live server, accessible to only a few system administrators
- Ongoing research on trusted applications
  - E.g. OS computes checksum on application to verify corruption
  - Allows file-system access only to trusted applications

# Protection from admin/super-users

- Operating system administrators (also known as super-users) can do anything they want to the database.
  - Small number of trusted administrators
- What if a laptop with critical data is lost?
  - Encrypt entire database (and/or file system)
    - Supported, e.g. in SQL Server 2005
    - Authentication (password/smart card) when database is started up

# Detecting Corruption

- Audit trails: record of all (update) activity on the database: who did what, when
  - Application level audit trail
    - Helps detect fraudulent activities by users
    - Independent audit section to check all updates
    - BUT: DBAs can bypass this level
      - E.g. audit trail apparently deleted in New Delhi auto-rickshaw license case by malicious users with DBA access
  - Database level audit trail
    - Database needs to ensure these can't be turned off, and turned on again after doing damage
    - Supported by most commercial database systems
    - But required DBAs with knowledge of application to monitor at this level
  - Keep archival copies and cross check periodically

# Summary

- Data security is critical
- Requires security at different levels
- Several technical solutions
- But human training is essential

# References


- Chapter 8 of Database System Concepts 5th Edition, Silberschatz, Korth and Sudarshan, McGraw-Hill
- The Open Web Application Security Project
  - http://www.owasp.org
- Web application security scanners
  - e.g. WebInspect (SPI Dynamics)
  - http://www.windowsecurity.com/software/Web-Application-Security/
- SQL Injection
  - http://www.cgisecurity.com/development/sql.shtml
- 9 ways to hack a web app
  - http://developers.sun.com/learning/javaoneonline/2005/webtier/TS-5935.pdf
- Related research papers
  - Kabra, Ramamurthy and Sudarshan, Redundancy and Information Leakage in Fine-Grained Access Control, SIGMOD 2006
  - Rizvi, Mendelzon, Sudarshan and Roy, Extending Query Rewriting Techniques for Fine-Grained Access Control, SIGMOD 2004